

Running the unit tests

These are hand-written unit tests for `base64_encode()/base64_decode()` in the real, unmodified `neovim/src/nvim/base64.c` (v0.12.5) – the same function targeted by `../fuzzing/` and `../cbmc/`, so the project applies three independent techniques to one function and can compare what each buys. See `../ProjectAnalysisReport.md` (“Unit tests + code coverage”) for the full rationale and results.

Why not a test framework

`base64.c` only calls `xmalloc()/xfree()` and exposes two pure functions with no object/class state, so there is nothing for a class-oriented framework like the course’s `QtTest` to attach to. `tests/test_base64.c` is a plain C file with a small `assert()-style CHECK()` macro and a `main()` that runs every test function and reports a pass/fail count – the whole “framework” is about 15 lines.

Prerequisites

- A C compiler (`cc`) that supports `--coverage` (both `clang` and `gcc` work).
- `lcov/genhtml` (`brew install lcov` on macOS, `apt install lcov` on Debian/Ubuntu).
- The `neovim` submodule configured at least once, so the generated headers `test_base64.c/base64.c` depend on exist:

```
cd neovim && cmake --preset default && cd ..
```

(A full `cmake --build build` is not required just to run these tests – configuring is enough to generate the headers – but running the `fuzzing/` and `cbmc/` scripts, or building the actual `nvim` binary, does need it.)

Running

From the repository root:

```
python3 unit_tests/run_tests.py
```

This will, from scratch:

1. Compile `tests/test_base64.c` and `neovim/src/nvim/base64.c` with `--coverage`, into `unit_tests/build/`.
2. Link and run the resulting `test_base64` binary, printing all tests passed (exit code 0) or a list of failed assertions (non-zero exit code).
3. Run `lcov --capture` over `unit_tests/build/`, then `lcov --extract` to scope the report to `base64.c` alone (the raw capture also picks up whatever headers the compiler pulled in, which isn’t the code under test).
4. Render `unit_tests/coverage_html/index.html` with `genhtml`.

What the test file covers

- **Known-vector tests** (`test_known_vectors`): encode/decode against the standard RFC 4648 base64 examples (“Man” -> “TWFu”, etc.) – an oracle independent of the implementation, not just a round trip that could hide a symmetric encode/decode bug.

- **Round-trip tests** (`test_roundtrip_all_short_lengths`): every input length from 0 to 32 bytes, to exercise the 8-byte and 4-byte bulk-copy loops in `base64_encode()` plus all three tail-remainder branches (0, 1, or 2 leftover bytes).
- **Rejection tests**: a length that isn't a multiple of 4, an invalid alphabet character, a misplaced `=`, and a wrong padding count – the four distinct ways `base64_decode()` takes its invalid: exit path.

Result (last run against commit 5885a30e1e1225349079e7a1c4a3848aa8e43e42)

All 6 test functions pass. Coverage of `base64.c` itself:

Metric	Result
Lines	98.1% (106/108)
Functions	100% (2/2)
Branches	85.2% (46/54)

For comparison, upstream's own test/unit suite gets **0%** line coverage on this file (see `../ProjectAnalysisReport.md` section 1) – it never exercises `base64.c` at all. The uncovered lines/branches are almost entirely the `goto invalid` arithmetic-edge-case checks (e.g. `acc_len > 4`) that are hard to hit without also being caught by an earlier check; full report: `coverage_html/index.html`.